

COMP 304:

Project 3 for Coffee Lovers

Due: 15/05/2026 23:59

Didem Unat, Spring 2026

Notes: This assignment **must be** completed in **teams of 2**. You may discuss the problems with other students, but the submitted work must be your team's own work. **Any material you use from the web (or an AI model) should be properly cited in your submission. Any sort of cheating, including directly copy-pasting AI-generated code, will be harshly PUNISHED.**

This assignment is worth 20% of your total grade (10% for the implementation and invited demos, and 10% for the project exam). The project exam will be conducted during the final exam. To be eligible to take the project exam, you must complete and submit the assignment before the deadline. We strongly recommend that you start early.

Responsible TAs:

Name	Email	Office	Office Hours
Doğan Sağbılı	dsagbili17@ku.edu.tr	ENG 230	Mondays 4:30-6:00 PM

Description

In this project, you will get more familiar with the concepts of scheduling, synchronization, multi-threading, and deadlock prevention in operating systems by using the POSIX threads (pthreads) API.

Coffee Supply Chain Simulator

You are an Operations Manager at Kuffee, an international coffee logistics company. Kuffee wants to observe the current maritime supply chain for its operations by simulating freight, production hubs, maritime trade routes, chokepoints, and coffee consumers.

You are assigned to implement a simplified model of the maritime supply chain and simulate the disruptions in the maritime chokepoints. The simulator should include the modeling of hubs producing coffee, maritime trade routes through straits, and consumers. It is crucial to ensure that the simulation implementation is **race-free** and **deadlock-free** to prevent any miscounts or disruptions and must work in **any map or simulation setup**.

The assignment has 5 main parts (100 points). Please note that some students may be randomly selected for in-person demos. Points may be deducted for poor performance during the demo.

Here are the rules for the simulation environment:

- The simulation should run in real-time. The simulation time is in seconds. Start from 0 and simulate until the simulation time reaches T_{max} .
- There are F many freights available in the system at a time ; a freight $f \in F$ carries a coffee order from a selected production hub p_f to a selected consumer c_f with quality $q_f \in \{low = 1, mid = 2, high = 3\}$ and amount $s_f \leq 1, s_f \geq S_{max}$. When a freight arrives at the consumer c_f , it exits the system, and a new freight is entered into the system.
- There are P many production hubs in the system. Each production hub $p \in P$ creates coffee supply with a $0 \leq x_p \leq 1$ probability per second, at a random quality level $q_p \in \{low = 1, mid = 2, high = 3\}$, and random amount s_p between 1 and S_{max} .
- Each coffee supply created from production hubs is added to a global production list, from which a freight can pick up.
- There are C many consumers in the system. A consumer $c \in C$ demands coffee with a $0 \leq x_c \leq 1$ probability, a random quality level $q_c \in \{low = 1, mid = 2, high = 3\}$, and an random amount s_c between 1 and S_{max} .
- Each coffee demand from consumers is placed on a global demand list, from which a freight can select for delivery.
- There are K many straits that a freight can travel through. A strait $k \in K$ has a travel time of t_k and only **one** freight can pass through at a time during its travel through the strait.
- There is a global map $M = (V, E)$ that describes the routes required for each production hub-consumer pair.
 - V is the set of locations. The set of locations V is partitioned into three distinct subsets: Producer hubs (P), Consumers (C), and Straits (K).
 - E is the set of routes where $e = (u, v) \in E$ where $u, v \in V$ representing the path between two vertices. Each route has a time of travel t_e .
- A freight can only move the coffee supply between a production hub p and a consumer c if there are matching coffee supply and demand from the global production and demand lists. The criteria for this match is: $q_p \geq q_c$ and $s_p \geq s_c$
- For efficiency, a strait decides which freight to pass, such that for any two freight, it favors the freight with higher quality coffee. If the coffee quality is the same among two freights, a strait favors the freight with higher amount of coffee..
- A freight f that carries a coffee order from a selected production hub p to a selected consumer c must select the shortest route based on the predetermined travel time of routes t_e .

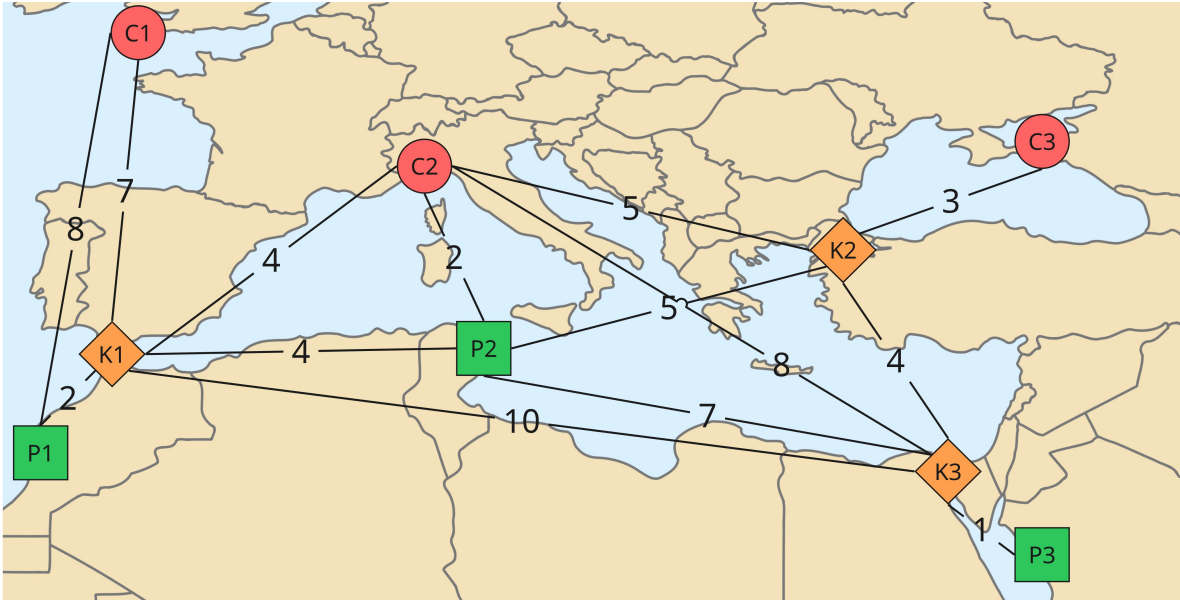


Figure 1: The global map example that is going to be used for this project. Green squares are production hubs P , red circles are consumers C , and orange diamonds are straits K . The black lines represent the set of routes E , with each number on a line representing the travel time t_e .

Simulation Setup

You must use predefined parameters and the map in Figure 1 to get the logs of the simulation. Parameters are: $T_{max} = 60$, $S_{max} = 5$, $F = 6$, $x_{p_1} = 0.9$, $x_{p_2} = 0.85$, $x_{p_3} = 0.75$, $x_{c_1} = 0.5$, $x_{c_2} = 0.9$, $x_{c_3} = 0.7$, $t_{k_1} = 3$, $t_{k_2} = 5$, $t_{k_3} = 7$

Keeping logs

You must keep a log of all events created by all entities that occur within the simulation, which will help you debug and test your code. The threads simulating entities should not keep logs; otherwise, logging may affect the simulation. To prevent such a situation, you should use the main thread or separate threads for logging.

- (10 points) The freight.log should keep the freight no. (ID), its starting point p_f and destination c_f , the production and demand IDs, the coffee order's quality q_f and amount s_f , and the arrival and departure times of the system.
- (15 points) The strait.log should keep all strait events within the simulation: the strait ID, the freight no. (ID) that entered the strait at what time, and which freights are currently waiting while a freight passes.
- (15 points) The coffee.log should track the status of the global production and demand lists, including each production/demand item's ID, when it is added, coffee quality, and quantity. The log should update when a new supply/demand is added to from the respective list, and a freight with ID when removing a supply and demand pair from both lists.

Part I

(20 points) To start small, implement the simulation with only one production hub P_1 , one strait K_1 , and one consumer C_2 with the routes between them as shown in the Figure 1. Get the logs for this simulation.

Part II

(15 points) To increase complexity, add one more consumer, C_1 , and any related additional routes to the simulation, based on the given map in the Figure 1. Get the logs for this simulation separately from Part I .

Part III

(15 points) To increase complexity from Part II, add one more production hub, P_2 , and any related additional routes to the simulation, based on the given map in the Figure 1. Get the logs for this simulation separately from Parts I and II.

Part IV

(10 points) To finalize, extend your implementation from Part III to add the remaining entities to the simulation using the map shown in Figure 1. Get the logs for this simulation separately from Parts I, II, and III.

Deliverables

Submit your assignment to LearnHub as a **single zip file** named `username1_username2.zip`.

Your submission must include:

- **Source Code:** A copy of the latest cloned version of your Git repository.
- **Log Files:** Each part should have their logs stored within a directory.
- **A README file:** This must include a link to your GitHub repository, a detailed description of your implementation and how to build and run it.

Your submission must NOT include any executable files or object files (.o)

GitHub Workflow Requirements:

- **Repository Setup:** Join the Github Classroom at <https://classroom.github.com/a/a5mEEoaA> and make sure there is a repository for your group.
- **Commit History:** You must make consistent, incremental commits from start to finish so we can track your progress. Do not commit the entire assignment at once at the very end; doing so may result in a grading penalty. We will review your commits during the assignment evaluation.

General Rules & Grading:

- **Environment:** You are required to implement the assignment on a Linux environment.
- **Interviews:** Selected teams may perform **in-person interview** after the project submission deadline. Both team members must be fully knowledgeable about the entire implementation. If you cannot adequately explain your code or demonstrate a clear understanding of the concepts, **you may receive zero** for the corresponding part of the project.

Good Luck.